

(72 points possible)

✓ 1) Logic (10 pts, 2 each)

A program has three Boolean variables, `error`, `discount`, and `pro`. For each verbal description of when we want a value of True, code the expression that is True exactly when the description is True and False otherwise. For example, "Error and discount are both True, and pro doesn't matter" dictates an answer of `error and discount`.

a) `error`, `discount`, and `pro` are all True.

```
# TODO
error and (discount and pro)
```

b) At least one of `error`, `discount`, and `pro` is True.

```
# TODO
error or (discount or pro)
```

c) `pro` doesn't matter, but either `error` is True or `discount` is False.

```
# TODO

error or discount
```

d) None of the three variables is True.

```
# TODO

!(error and (discount and pro))
```

e) At least two of the three variables are True.

```
# TODO
```

```
(error and discount) or ((discount and pro) or (pro and error))
```

2) Importing modules (5 pts [3, 2])

Answer the following questions.

a, 3pts) In the code cell below you are given a list of numbers. Using the keyword `from`, import the following functions from the math module.

```
floor  
ceil  
sqrt
```

Iterate over the given list of numbers and print "sqrt(X) is between Y and Z" for each, where X is the number from the list, Y is the floor of the square root, and Z is the ceiling of the square root; for example, "sqrt(3.5) is between 1 and 2."

```
numbers = [3.5, 10.1, 4.7, 8.9, 16.0, 23.2]  
# TODO  
from math import sqrt, floor, ceil  
  
for i in numbers:  
    rooted = sqrt(i)  
    flr = floor(rooted)  
    cil = ceil(rooted)  
    print(f"sqrt({i}) is between {flr} and {cil}.")
```

```
sqrt(3.5) is between 1 and 2.  
sqrt(10.1) is between 3 and 4.  
sqrt(4.7) is between 2 and 3.  
sqrt(8.9) is between 2 and 3.  
sqrt(16.0) is between 4 and 4.  
sqrt(23.2) is between 4 and 5.
```

b, 2pt) Import `string`, then print the letters of the alphabet in order (one letter per line) using `string.ascii_lowercase`, a string that contains each lowercase letter once. Note that you can iterate over the characters of a string as if it were a list.

```
# TODO
import string as str

for i in str.ascii_lowercase:
    print(i)
```

```
a
b
c
d
e
f
g
h
i
j
k
l
m
n
o
p
q
r
s
t
u
v
w
x
y
z
```

3) Looping over names (30 pts [5, 5, 8, 7, 5])

You are considering baby names for your children - you expect to have two. The names you've narrowed it down to are:

```
[
    "Zephyrus", "Clementine", "Hendrixon", "Quintavius", "Maverick", "Xanth",
    "Bellatrix", "Orionis", "Jupiter", "Vespera", "Djangolina", "Ravenwood",
    "Echolette", "Sableine", "Peregrine", "Wrenly", "Ivyanna", "Lunaria",
    "Fitzgerald", "Gizmonda", "Axelrod", "Nashira", "Yaralyn", "Talonis",
    "Kodamine", "Umaira"
]
```

a, 5 pts) Write a loop that prints out all possible combinations of two baby names, one per line, such as "Zephyrus-Umaira". Do not repeat the same name on the same line. You can repeat the same combination in a different order, however, since the first name will be the name of the older child.

```
babynames = [
    "Zephyrus", "Clementine", "Hendrixon", "Quintavius", "Maverick", "Xa",
    "Bellatrix", "Orionis", "Jupiter", "Vespera", "Djangolina", "Ravenwo",
    "Echolette", "Sableine", "Peregrine", "Wrenly", "Ivyanna", "Lunaria",
    "Fitzgerald", "Gizmonda", "Axelrod", "Nashira", "Yaralyn", "Talonis",
    "Kodamine", "Umaira"
]
# TODO
for i in range(len(babynames)):
    for ii in range(i, len(babynames)):
        if(babynames[i] != babynames[ii]):
            print(babynames[i] + "-" + babynames[ii])
```

```
Zephyrus-Clementine
Zephyrus-Hendrixon
Zephyrus-Quintavius
Zephyrus-Maverick
Zephyrus-Xanthippe
Zephyrus-Bellatrix
Zephyrus-Orionis
Zephyrus-Jupiter
Zephyrus-Vespera
Zephyrus-Djangolina
Zephyrus-Ravenwood
Zephyrus-Echolette
```

Zephyrus-Sableine
Zephyrus-Peregrine
Zephyrus-Wrenly
Zephyrus-Ivyanna
Zephyrus-Lunaria
Zephyrus-Fitzgerald
Zephyrus-Gizmonda
Zephyrus-Axelrod
Zephyrus-Nashira
Zephyrus-Yaralyn
Zephyrus-Talonis
Zephyrus-Kodamine
Zephyrus-Umaira
Clementine-Hendrixon
Clementine-Quintavius
Clementine-Maverick
Clementine-Xanthippe
Clementine-Bellatrix
Clementine-Orionis
Clementine-Jupiter
Clementine-Vespera
Clementine-Djangolina
Clementine-Ravenwood
Clementine-Echolette
Clementine-Sableine
Clementine-Peregrine
Clementine-Wrenly
Clementine-Ivyanna
Clementine-Lunaria
Clementine-Fitzgerald
Clementine-Gizmonda
Clementine-Axelrod
Clementine-Nashira
Clementine-Yaralyn
Clementine-Talonis
Clementine-Kodamine
Clementine-Umaira
Hendrixon-Quintavius
Hendrixon-Maverick
Hendrixon-Xanthippe
Hendrixon-Bellatrix
Hendrixon-Orionis
Hendrixon-Jupiter
Hendrixon-Vespera
Hendrixon-Djangolina
Hendrixon-Ravenwood
Hendrixon-Echolette
Hendrixon-Sableine

b, 5 pts) Well, that's too many combinations to consider. You decide to limit the search to names that have at least two "e"'s to make it sound more mellifluous. Use a for loop to create a list of such names. You can use the `.count()` string method to figure out whether a name has enough e's (`s.count(n)` returns the number of times `n` appears in `s`). Check for both uppercase and lowercase. You don't need to do the nested loop again - just print the short list of two-e names. (Be sure it's also assigned to a variable so that you can do the next part.)

```
# TODO
namesWithEs=[]
for i in babynames:
    if(i.lower().count("e")>=2):
        namesWithEs.append(i)
```

c, 8 pts) Wait - middle names! Print all combinations of two different names followed by two other different names, using your shorter list of names that have enough e's. For example, "Xanthippe Yaralyn-Xanthippe Umaira" refers to two sisters who both have a first name of Xanthippe. As this example suggests, you can now repeat first names between individuals, since we have the middle names to distinguish them, but no individual should get a middle name that is the same as their first name (no "Xanthippe Xanthippe") and the two individuals should not have identical name combinations (no "Xanthippe Yaralyn-Xanthippe Yaralyn"). Tip: You might find it useful to write the first draft of the code without quite so many rules.

```
# TODO

def namegen():
    retval = []
    for i in range(len(namesWithEs)):
        for ii in range(i, len(namesWithEs)):
            if(babynames[i]!=babynames[ii]):
                retval.append(namesWithEs[i]+" "+namesWithEs[ii])
    return retval

sis1 = namegen()
sis2 = namegen()

for i in sis1:
```

```
for ii in sis2:
    if (i!=ii):
        print(i,"-",ii)
```

Clementine Vespera – Clementine Echolette
Clementine Vespera – Clementine Sableine
Clementine Vespera – Clementine Peregrine
Clementine Vespera – Vespera Echolette
Clementine Vespera – Vespera Sableine
Clementine Vespera – Vespera Peregrine
Clementine Vespera – Echolette Sableine
Clementine Vespera – Echolette Peregrine
Clementine Vespera – Sableine Peregrine
Clementine Echolette – Clementine Vespera
Clementine Echolette – Clementine Sableine
Clementine Echolette – Clementine Peregrine
Clementine Echolette – Vespera Echolette
Clementine Echolette – Vespera Sableine
Clementine Echolette – Vespera Peregrine
Clementine Echolette – Echolette Sableine
Clementine Echolette – Echolette Peregrine
Clementine Echolette – Sableine Peregrine
Clementine Sableine – Clementine Vespera
Clementine Sableine – Clementine Echolette
Clementine Sableine – Clementine Peregrine
Clementine Sableine – Vespera Echolette
Clementine Sableine – Vespera Sableine
Clementine Sableine – Vespera Peregrine
Clementine Sableine – Echolette Sableine
Clementine Sableine – Echolette Peregrine
Clementine Sableine – Sableine Peregrine
Clementine Peregrine – Clementine Vespera
Clementine Peregrine – Clementine Echolette
Clementine Peregrine – Clementine Sableine
Clementine Peregrine – Vespera Echolette
Clementine Peregrine – Vespera Sableine
Clementine Peregrine – Vespera Peregrine
Clementine Peregrine – Echolette Sableine
Clementine Peregrine – Echolette Peregrine
Clementine Peregrine – Sableine Peregrine
Vespera Echolette – Clementine Vespera
Vespera Echolette – Clementine Echolette
Vespera Echolette – Clementine Sableine
Vespera Echolette – Clementine Peregrine
Vespera Echolette – Vespera Sableine
Vespera Echolette – Vespera Peregrine
Vespera Echolette – Echolette Sableine
Vespera Echolette – Echolette Peregrine
Vespera Echolette – Sableine Peregrine
Vespera Sableine – Clementine Vespera
Vespera Sableine – Clementine Echolette

```
Vespera Sableine - Clementine Sableine  
Vespera Sableine - Clementine Peregrine  
Vespera Sableine - Vespera Echolette  
Vespera Sableine - Vespera Peregrine  
Vespera Sableine - Echolette Sableine  
Vespera Sableine - Echolette Peregrine  
Vespera Sableine - Sableine Peregrine  
Vespera Peregrine - Clementine Vespera  
Vespera Peregrine - Clementine Echolette  
Vespera Peregrine - Clementine Sableine  
Vespera Peregrine - Clementine Peregrine  
Vespera Peregrine - Vespera Echolette
```

d, 7 pts) How are you going to decide between all these combinations? Well, you still like the letter 'e', you decide. Copy your code from the last section to iterate through all the combinations of two names with two other names (again just using names with at least two e's), but this time, keep track of which combination has the most e's (uppercase or lowercase) between all four names. Print the number of e's at the end of every line, and finally print "Best names: " followed by the combination with the most e's. In case of a tie, you only need to print one of the best combinations.

```

# TODO
# TODO

def namegen():
    retval = []
    for i in range(len(namesWithEs)):
        for ii in range(i, len(namesWithEs)):
            if(babynames[i] != babynames[ii]):
                retval.append(namesWithEs[i] + " " + namesWithEs[ii])
    return retval

sis1 = namegen()
sis2 = namegen()

allNames = []
for i in sis1:
    for ii in sis2:
        if (i != ii):
            allNames.append((i + " - " + ii, (i + ii).upper().count("E")))

maxEs = 0
for i, ii in allNames:
    if(ii > maxEs):
        maxEs = ii
print("Best Names:")
for i, ii in allNames:
    if(ii == maxEs):
        print(i)

```

```

Best Names:
Clementine Echolette - Clementine Peregrine
Clementine Echolette - Echolette Peregrine
Clementine Peregrine - Clementine Echolette
Clementine Peregrine - Echolette Peregrine
Echolette Peregrine - Clementine Echolette
Echolette Peregrine - Clementine Peregrine

```

e, 5 pts) What are the names likely to cause ties here? Write a list comprehension that creates a list of tuples of the form (name, count), where name is the baby name and count is the number of 'e' and 'E' characters in it. Print the list you made.

```

# TODO
#but... I already did that...

def namegen():
    retval = []
    for i in range(len(namesWithEs)):
        for ii in range(i, len(namesWithEs)):
            if(babynames[i] != babynames[ii]):
                retval.append(namesWithEs[i] + " " + namesWithEs[ii])
    return retval

sis1 = namegen()
sis2 = namegen()

allNames = []
for i in sis1:
    for ii in sis2:
        if (i != ii):
            allNames.append((i + " - " + ii, (i + ii).upper().count("E")))

```

4) Testing and debugging (15 pts: 5, 10)

a 5 pts) Your colleague Barabbas comes to you with the following snippet of Python code that he says is not working. He wants the code to countdown to zero when given a positive number. Correct Barabbas's code in the empty cell below. Be aware that there are multiple errors.

```

# Barabbas's incorrect code

value = input('Enter a positive number: ')
while value < 0
    print(value)
    value = value - 1

```

```
# TODO
```

```
value = int(input('Enter a positive number: '))  
while (value >= 0):  
  
    print(value)  
    value = value - 1
```

b 10 pts) After fixing his code, Barabbas comes for help again. He has a list

```
my_integers = [3, 4, 5, 6, 11, 120, -10, 1]
```

and he is trying to check if an input number is not in the list. If the input number is not in the list he wants to print a single line of output that says `True, not in the list`. Otherwise the code should output a single line `False, in the list`. The current code only gives errors. Fix the code so that there are no more errors and so that it produces the correct behavior.

```
# Barabbas's incorrect code
```

```
my_integers = [3, 4, 5, 6, 11, 120, -10, 1]  
not_in_the_list = True  
value = input("Check if this number is not in the list: ")  
for i in my_integers:  
    if value == i:  
        print(not_in_the_list + " not in the list" )  
    else:  
        print(not_in_the_list + " in the list")
```

```
# TODO
my_integers = [3, 4, 5, 6, 11, 120, -10, 1]
not_in_the_list = True
value = int(input("Check if this number is not in the list: "))
for i in my_integers:

    if(value ==i):
        not_in_the_list=False

if not_in_the_list == True:
    print(not_in_the_list , "not in the list" )
else:
    print(not_in_the_list , "in the list")

False in the list
```

5) Iteration (10 pts [4, 1, 4, 1])

a 4 pts) You are given the following lines of code

```
print(count)
count = 0
count += 1
for i in range(0, n):
    for j in range(0, n):
        for k in range(0, n):
            n = # TODO
```

Correctly set the value of n and arrange and indent these lines of code so that you print a value of 64 at the end of the execution.

```
# TODO
n = 4
count = 0

for i in range(0, n):
    for j in range(0, n):
        for k in range(0, n):
            count += 1

print(count)
```

64

b 1 pt) Consider what happens when we assign different values to n . How does the value of count depend on n -- is it equal to n , n^2 , n^3 , 2^n or something else? (You can click on this textbox to see how to format text with exponents.)

Answer: **TODO**

n is the base of the exponential number, and the number of nested for loops that loop for n times is the exponent only if the most inner statement is adding 1 to an int variable assigned 0.

c 4 pts) You are given the following lines of code

```
print(count)
count = 1
count *= 2
for i in range(0, n):
    n = # TODO
```

Correctly set the value of n and arrange these lines of code so that you print a value of 32 when you execute the code.

```
# TODO
count = 1
n = 5;
for i in range(0,n):
    count*=2
print(count)
```

32

d 1 pt) How does the value of count depend on n ?

n here is the exponential value of 2. (2^n)

Answer: **TODO**

✓ AI Statement (2 points)

Please put an X in the box that best matches your AI use for this homework ([X] in the markup will become a checked box). There is no penalty for using AI, but we want to understand how you're using it.

- ☒ I did not use AI to complete this assignment.
- ☐ I didn't use AI to debug or generate code, but I did use it as a Python reference.
- ☐ I didn't use AI to generate code, but I did use it to debug some code.
- ☐ I did use AI to generate some code, but not all the code for any problem.
- ☐ I used AI to solve at least one problem completely.

When you are done, submit both your .ipynb (File->Download->Download .ipynb in Google Colab) and a PDF (File->Print->Save to PDF) to Blackboard where you found this assignment. (The backup PDF helps us give you points if there's a problem with your .ipynb.)

